MANUAL TÉCNICO

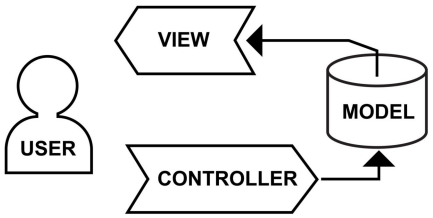
1. Introducción y Objetivos del Sistema

El presente documento detalla la arquitectura, decisiones de diseño y algoritmos principales del sistema de simulación y gestión de pizzerías. Este software fue desarrollado utilizando **Java** y **Java Swing** para la interfaz gráfica.

El objetivo principal es simular un entorno de trabajo de alta presión mientras se mantiene un módulo administrativo robusto para la gestión de sucursales, usuarios y reportería. El sistema está diseñado para ser escalable, seguro y altamente modular.

2. Arquitectura de Software

Para garantizar un código limpio y mantenible, el sistema se construyó bajo el patrón arquitectónico **MVC (Modelo-Vista-Controlador)** puro, apoyado por inyección de dependencias.



MODEL - VIEW - CONTROLLER PATTERN

Capa Lógica (Modelos): Clases planas (POJOs) que representan las entidades del negocio (`Usuario`, `Pedido`, `SucursalProducto`, `Partida`).

Capa de Presentación (Vistas): Componentes visuales (`JFrame`, `JPanel`, `JDialog`) que carecen de lógica de negocio. Solo declaran elementos gráficos y exponen *Getters* para que el controlador los manipule.

Capa de Control (Controladores): Actúan como puente. Suscriben los eventos (`ActionListeners`) de la Vista, procesan las reglas del negocio y coordinan la persistencia de datos.

2.1 Patrones de Diseño Adicionales

DAO (Data Access Object): Aísla las consultas SQL (`INSERT`, `UPDATE`, `SELECT` con `JOIN`) del resto de la aplicación.

Singleton: Aplicado en la clase `Conexion` para garantizar que toda la aplicación comparta una única instancia de conexión a la base de datos MySQL, optimizando el uso de memoria.

3. Algoritmos Core y Lógica de Negocio

El sistema incluye algoritmos de simulación que evalúan variables en tiempo real.

3.1 Motor de Generación de Pedidos Aleatorios

El generador simula el comportamiento humano al realizar un pedido, garantizando que no se rompan las reglas de integridad.

> **Lógica de Selección:** El sistema utiliza `Random` para definir un límite en la cantidad total de pizzas solicitadas.
>
> **Distribución Aleatoria:** Se emplea `Random` sobre la lista del menú para barajar las opciones disponibles, asi todos los productos del menu puedan ser elejidos de manera alteatoria.

3.2 Balanceo Dinámico de Dificultad (Time-Scaling)

El tiempo límite de un pedido no es estático. El algoritmo calcula el tiempo total iterando sobre los productos en el "carrito" virtual:

> Verifica si el producto específico tiene un tiempo de preparación propio. Si es `0`, hereda el tiempo base configurado en el sistema.
>
> Evalúa el nivel actual del jugador. Si está en Nivel 2 o 3, se aplica una resta multiplicativa basada en el parámetro `dificultad_nivel`, reduciendo el tiempo disponible y aumentando el estrés del juego.
>
> Incluye un candado de seguridad que evita que el cálculo matemático resulte en tiempos negativos o injugables (estableciendo un límite inferior de 10 segundos por pizza).

3.3 Concurrencia, Timers y Estado de la Cocina

El juego maneja múltiples hilos simultáneos mediante `Timer`.

> **Relojes Individuales:** Cada `PanelPedidoCard` posee su propio Timer independiente que actualiza la UI y evalúa condiciones de penalización si el tiempo llega a cero (`No_Entregado`).
>
> **Reloj Maestro:** Un Timer global administra el "Turno de Trabajo" (150 segundos). Al finalizar, detiene el generador de clientes, ejecuta un barrido sobre el contenedor de componentes gráficos (`getComponents()`), evalúa qué tarjetas no están completadas, aplica penalizaciones en bloque y sella la partida en la base de datos.

Prevención de NullPointerException: Se implementaron validaciones que abortan la ejecución de los Timers individuales si detectan que la instancia de `PartidaActual` ya ha sido destruida por el reloj maestro.

4. Estructura de Base de Datos y Persistencia

El sistema utiliza MySQL con motor InnoDB para garantizar la integridad referencial.

Las contraseñas de los usuarios no se guardan en texto plano, sino que están preparadas para procesos de encriptación.

Se utiliza un modelo transaccional. La inserción de un `pedido` devuelve su `AUTO_INCREMENT` generado (`Statement.RETURN_GENERATED_KEYS`), el cual es inmediatamente inyectado en un bucle para insertar los múltiples registros en la tabla `detalle_pedido`.